

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

Кафедра вычислительной техники

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Организация процессов и программирование в среде
Linux»
Тема: Обработка сигналов

Студент гр. 3311 _____

Баймухамедов Р. Р.

Преподаватель _____

Лазарев Э. А.

Санкт-Петербург

2026

ЦЕЛЬ РАБОТЫ

Познакомиться с механизмом сигналов POSIX и способами их обработки в программе Linux.

ЗАДАНИЕ

1. Написать программу, которая реагирует на ошибку деления на ноль и на нарушение защиты памяти при обращении через неверный указатель. Тип ошибки передаётся аргументом командной строки. Обработчик должен вывести сообщение и завершить программу с кодом 1 или 2.

2. Откомпилировать программу и запустить её дважды: отдельно для каждого типа ошибки.

1 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Сигнал сообщает процессу о событии. На учебной системе деление на ноль вызывает SIGFPE, обращение по недопустимому адресу — SIGSEGV. По умолчанию оба завершают процесс. `sigaction` задаёт обработчик в `sa_handler`; `sa_mask` и `sa_flags` настраивают его вызов.

Обработчик вызывается асинхронно. Для вывода и завершения с кодом в нём применяются безопасные функции `write` и `_exit`. `printf` и `exit` в обработчике недопустимы. Оболочка сохраняет код завершения в `$?`.

2 ПРОГРАММА

2.1 Принцип работы

Программа принимает аргумент `div0` или `ptr`, устанавливает обработчики SIGFPE и SIGSEGV и выполняет выбранную ошибочную операцию. Обработчик выводит сообщение и кодом завершения обозначает тип ошибки. Перед операцией стандартный вывод сбрасывается для корректного отображения в терминале.

2.2 Текст программы

```
#define _POSIX_C_SOURCE 200809L
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
static void handle_fault(int signo)
{
    if (signo == SIGFPE) {
        const char msg[] = "Ошибка деления на ноль: SIGFPE\n";
        (void)write(STDERR_FILENO, msg, sizeof(msg) - 1);
        _exit(1);
    }
    if (signo == SIGSEGV) {
        const char msg[] = "Ошибка обращения к памяти: SIGSEGV\n";
        (void)write(STDERR_FILENO, msg, sizeof(msg) - 1);
        _exit(2);
    }
    _exit(3);
}
int main(int argc, char *argv[])
{
    if (argc != 2 ||
        (strcmp(argv[1], "div0") != 0 &&
         strcmp(argv[1], "ptr") != 0)) {
        fprintf(stderr, "Использование: %s div0|ptr\n", argv[0]);
        return EXIT_FAILURE;
    }
    struct sigaction action = {0};
    action.sa_handler = handle_fault;
    if (sigemptyset(&action.sa_mask) == -1 ||
        sigaction(SIGFPE, &action, NULL) == -1 ||
        sigaction(SIGSEGV, &action, NULL) == -1) {
        perror("sigaction");
        return EXIT_FAILURE;
    }
    if (strcmp(argv[1], "div0") == 0) {
        puts("Проверка деления на ноль");
        fflush(stdout);
        volatile int dividend = 10, zero = 0;
        volatile int result = dividend / zero;
        (void)result;
    } else {
        puts("Проверка недопустимого указателя");
        fflush(stdout);
        volatile int *ptr = NULL;
        *ptr = 42;
    }
    return EXIT_SUCCESS;
}
```

3 КОМПИЛЯЦИЯ И ЗАПУСК ПРОГРАММЫ

3.1 Компиляция

```
user@linuxlabs:~/lab05$ gcc signals.c -o signals
user@linuxlabs:~/lab05$ ls
signals  signals.c
user@linuxlabs:~/lab05$ ls -la
total 32
drwxrwxr-x  2 user user  4096 Sep 29 19:12 .
drwxr-x--- 10 user user  4096 Sep 29 18:11 ..
-rwxrwxr-x  1 user user 16480 Sep 29 19:12 signals
-rw-rw-r--  1 user user  1254 Sep 29 19:10 signals.c
user@linuxlabs:~/lab05$ _
```

Рисунок 1 - Компиляция программы

3.2 Деление на ноль

```
user@linuxlabs:~/lab05$ ./signals div0; echo $?
Check division by zero
Error division by zero: SIGFPE
1
```

Рисунок 2 - Обработка ошибки деления на ноль

3.3 Недопустимый указатель

```
user@linuxlabs:~/lab05$ ./signals ptr; echo $?
Check unallowable pointer
Error appeal to memory: SIGSEGV
2
user@linuxlabs:~/lab05$
```

Рисунок 3 - Обработка ошибки обращения к памяти

4 АНАЛИЗ РЕЗУЛЬТАТОВ

При div0 возникает SIGFPE и обработчик завершает процесс кодом 1. При ptr возникает SIGSEGV и код 2. Сообщение и код показывают тип ошибки.

Деление на ноль и запись по нулевому указателю в С имеют неопределённое поведение. Результат проверяется двумя запусками на учебной системе.

ВЫВОД

Изучена установка обработчиков с помощью sigaction. Программа выводит сообщение при SIGFPE и SIGSEGV и завершается кодом 1 или 2 соответственно.